

Oauth Session

Table of Contents

- OAuth And Session Documentation
 - Google OAuth Strategy
 - Google Console Origin Rules
 - Test/Development Mode
 - Session Strategy
 - Account Lifecycle
 - Manual Role-Test Login
 - Postman Session Verification
 - Back Button And Expiration Controls
 - Frontend Private Entry
 - Required Environment Variables

OAuth And Session Documentation

Google OAuth Strategy

1. Browser opens `/login.html`.
2. React calls `GET /api/v1/auth/config` to read the public Google OAuth client id.
3. Browser obtains a Google ID token through Google Identity Services.
4. Browser sends the ID token to `POST /api/v1/auth/google`.
5. Backend verifies the ID token with Google in production using `google-auth-library`.
6. Backend validates `sub`, `email`, verified email status and configured audience `GOOGLE_CLIENT_ID`.
7. Backend links `google_sub` only to an existing active internal user with the same verified email.

8. If the email is not already registered by the academy, Google login is rejected.
9. If an already linked Google subject now presents a different email, login is rejected to protect the academy-owned account identity.
10. Internal roles are assigned only through the application, mainly `POST /api/v1/users` and `PATCH /api/v1/users/{id}/role`.

Google Console Origin Rules

Google OAuth browser origins must use an exact allowed web origin. Deployed origins use the organization HTTPS domain; localhost is reserved for development. Never publish infrastructure addresses in the repository.

Recommended production origin:

- `https://<owned-domain>`

Temporary testing origin, only after DNS and HTTPS are configured:

- `https://academy.example.invalid` (documentation placeholder)

Google documentation reference:

<https://support.google.com/cloud/answer/15549257>

Test/Development Mode

`ALLOW MOCK GOOGLE TOKENS=true` or `NODE_ENV=test` allows signed mock JWT payloads for automated tests. Production must keep mock tokens disabled.

Session Strategy

- The application issues its own signed JWT session token after email/password or Google verification.
- `POST /api/v1/auth/login` is the normal email/password login for existing academy users with `users.password_hash`.
- `POST /api/v1/auth/google` returns the token as `data.sessionToken` with `data.tokenType = "Bearer"` for Postman/API verification.
- `POST /api/v1/auth/login` returns the same response shape and session behavior as Google login.
- The same token is also sent in cookie `alc_session` for browser/private page usage.
- Cookie options: HttpOnly, SameSite strict, Secure in production.
- Server persistence stores only a SHA-256 token hash, not the raw token.
- Session expiration defaults to `SESSION_TTL_MINUTES=120`.
- Authenticated API requests can use either cookie `alc_session` or `Authorization: Bearer <sessionToken>`.

- Logout revokes the server-side session hash and clears the cookie; the same Bearer token must return `401` after logout.
- New academy accounts default to `must_change_password=true`; protected academic endpoints return `403 PASSWORD_CHANGE_REQUIRED` until `/auth/change-password` succeeds.

Account Lifecycle

1. Admin or GeneralDirector creates the user with `POST /api/v1/users`.
2. The private React dashboard exposes the same "Create academy account" flow for Admin and GeneralDirector users.
3. BranchDirector account creation requires at least one assigned branch.
4. The email is the username for password login.
5. The backend stores only `password_hash`.
6. The director gives the user a temporary password.
7. The user signs in and must change the temporary password.
8. Google Sign-In can then be used as an alternate login when the Google email matches the academy account.

Manual Role-Test Login

Run `npm run db:seed:role-test` after the database schema is applied to create temporary users for Admin, GeneralDirector, BranchDirector, Teacher and Student. The seed stores only one-way password hashes in `users.password_hash`.

There are no default credentials. Supply local `SEED_*_EMAIL` and `SEED_*_PASSWORD` secrets explicitly; seeded accounts require a first-login password rotation. Mock Google tokens and the legacy Postman fallback are technically impossible outside tests.

Postman Session Verification

1. Run `Auth & Session / Current Session - No Token`; it must return `401`.
2. Run `Auth & Session / Password Login - Invalid Credentials`; it must return `401`.
3. Run `Auth & Session / Password Login - Demo`.
4. The Postman test stores `data.sessionToken` into `session_token`.
5. Run `Auth & Session / Current Session - Bearer Token`; it must return `200`.
6. Run `Session Teardown / Logout`; the backend revokes the persisted session hash.
7. Run `Session Teardown / Current Session - After Logout`; the same Bearer token must return `401`.

Back Button And Expiration Controls

- API and private pages send `Cache-Control: no-store`.
- `/private/*` is protected by session middleware.
- Missing/expired session redirects private HTML pages to `/login.html?session=expired`.
- Expired API sessions return `401`.

Frontend Private Entry

- Public landing access points to `/login.html`.
- Successful email/password or Google login redirects to `/private/dashboard.html`.
- First-login users see the mandatory password-change screen before the academic dashboard.
- Admin and GeneralDirector users can create academy accounts from the private dashboard, assign BranchDirector branches and see the one-time temporary password.
- Logout revokes the server-side session and redirects to `/login.html?session=logout`.
- The Content Security Policy allows the Google Identity Services script/frame while keeping application resources self-hosted.

Required Environment Variables

- `GOOGLE_CLIENT_ID`
- `SESSION_SECRET`
- `SESSION_TTL_MINUTES`
- `ALLOW MOCK GOOGLE TOKENS`
- `POSTMAN_LOGIN_ENABLED`
- `POSTMAN_LOGIN_EMAIL`
- `POSTMAN_LOGIN_PASSWORD`
- `SEED_ADMIN_PASSWORD`
- `SEED_GENERAL_DIRECTOR_PASSWORD`
- `SEED_BRANCH_DIRECTOR_PASSWORD`
- `SEED_TEACHER_PASSWORD`
- `SEED_STUDENT_PASSWORD`
- `CORS_ORIGINS`